

# zkEVM project 1

Execution Witness

Feb 2026





# engine\_newPayloadWithWitness exploration

- Goal: optimize witness generation flow within the EngineAPI
- Dimensions of exploration:
  - API design: “Enshrining” the execution witness in EngineAPI
  - Transport layer: JSON-RPC vs other potential options
- Results:
  - API design: explored in Geth, Reth and Ethrex with different conclusion for each
  - Transport layer: >20x perf improvement avoiding JSON-encoding using TCP.
- More details:
  - [EngineAPI witness generation report](#)
  - [TCP wire protocol for execution witness](#)

# SSZ EngineAPI



## Binary SSZ Transport for the Engine API: An initial benchmark on a live network with Kurtosis

■ Execution Layer Research



Giulio2002

7h

Mar 9

Author: Giulio Rebuffo

### Introduction

The [SSZ Engine API spec](#) proposes replacing JSON-RPC encoding with binary SSZ (Simple Serialize) for the Ethereum Engine API — the interface between consensus layer (CL) and execution layer (EL) clients.

Today, every message between CL and EL is JSON-encoded: every byte of every transaction, blob, and withdrawal is hex-encoded and wrapped in JSON. For small payloads like `forkchoiceUpdated`, this is fine. For `engine_getPayloadV5` — which returns the full `ExecutionPayload` plus `BlobsBundle` — the encoding overhead becomes a meaningful contributor to block propagation latency.

7h ago

## feat: add ssz to engine api #764

Draft

[barnabasbusa](#) wants to merge 11 commits into [ethereum:main](#) from [barnabasbusa:bbusa/ssz](#)



Conversation 1



Commits 11



Checks 5



Files changed 3



[barnabasbusa](#) commented 5 days ago • edited ▾

Member ...

Core change: Full binary SSZ over REST. No JSON, no hex encoding - raw SSZ bytes over HTTP.

Rationale:

1. Performance: Eliminates JSON parsing + hex encoding/decoding on the critical path. The CL already has execution payloads in SSZ (from beacon blocks) — with binary transport it can forward raw bytes to the EL without any conversion.
2. Blob scaling: As blob count increases, JSON becomes the bottleneck. Each blob is 131,072 bytes — hex encoding doubles that. Binary SSZ sends blobs at their actual size. This directly unblocks increasing `MAX_BLOB_COMMITMENTS_PER_BLOCK` without degrading CL-EL communication.
3. Zero breakage: JSON-RPC stays as default. SSZ is opt-in via capabilities exchange. Existing clients work unchanged.



7

[Binary SSZ Transport for the Engine API](#)

[feat: add ssz to engine api](#)



ignacio



kev



STEEL team

# Execution Witness specs

- Execution witness generation specs [released!](#)
  - Canonical witness generation logic present *execution-specs@projects/zkevm*
  - Contains filled ~12k tests with new *executionWitness* field in fixtures
- Specs were compared by working in parallel with Reth
  - Found multiple Reth discrepancies ([paradigmxyz/reth#22289](#), [paradigmxyz/stateless#8](#))
  - Found spec bugs which helped gaining more confidence for a stronger release
- Ask for action for ELs:
  - Run fixtures in stateful mode as usual, generate witness, and compare with *executionWitness* spec result
  - Discuss in existing EthR&D #l1-zkevm-protocol if you think there is a spec bug!
- Next steps: more thorough EEST tests to cover border cases

# zkEVM project 2

Guest Program

Feb 2026





ignacio



kev



STEEL team

# Guest program specs

- Guest program draft specs [released!](#)
  - Based on Amsterdam specs

```
def run_stateless_guest(input_bytes: Bytes) -> Bytes:
    """
    Run the stateless guest with serialized input, return serialized output.
    """
    stateless_input = deserialize_stateless_input(input_bytes)
    stateless_output = verify_stateless_new_payload(stateless_input)

    output_bytes = serialize_stateless_output(stateless_output)
    return output_bytes
```

- Next steps:
  - Polish implementation
  - Create more thorough tests to harden implementation

# zkEVM project 6

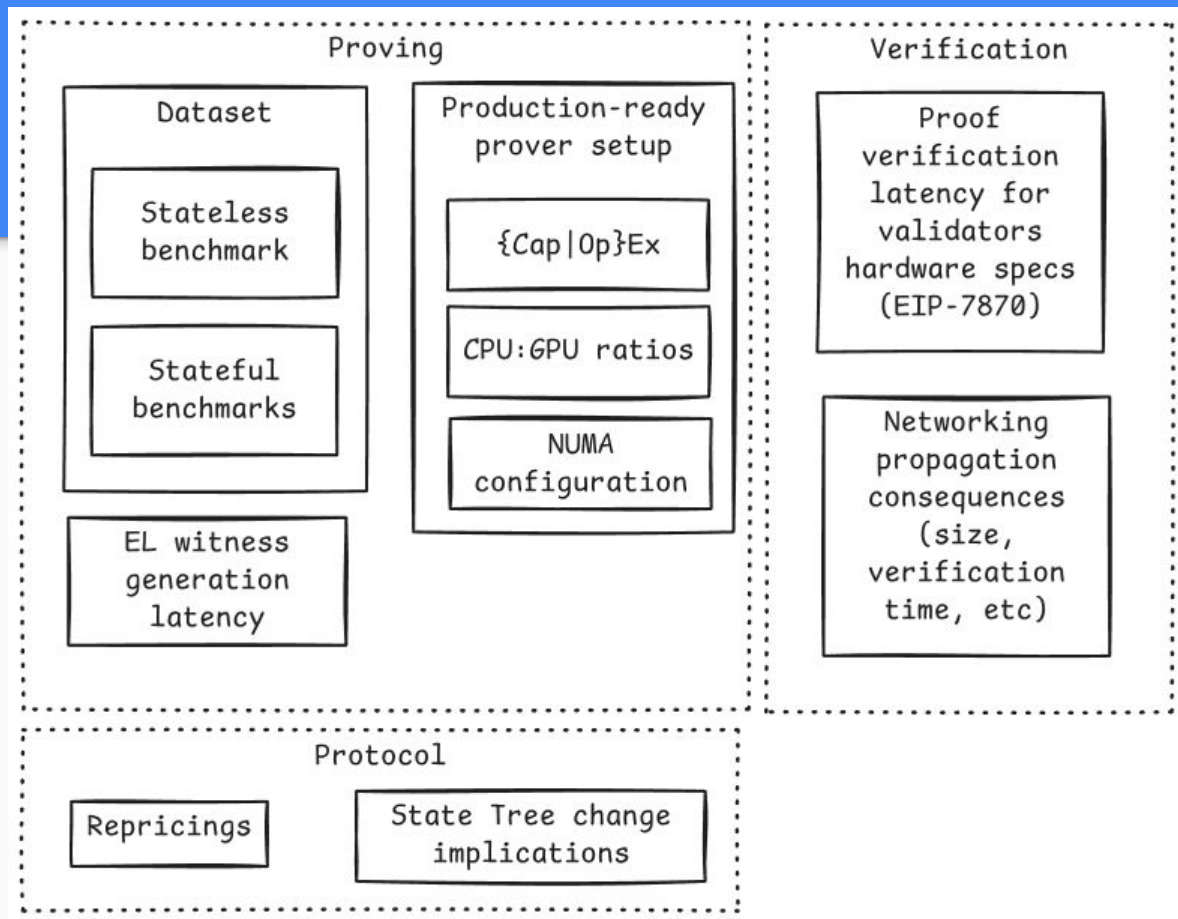
Benchmarking and Metrics

Feb 2026



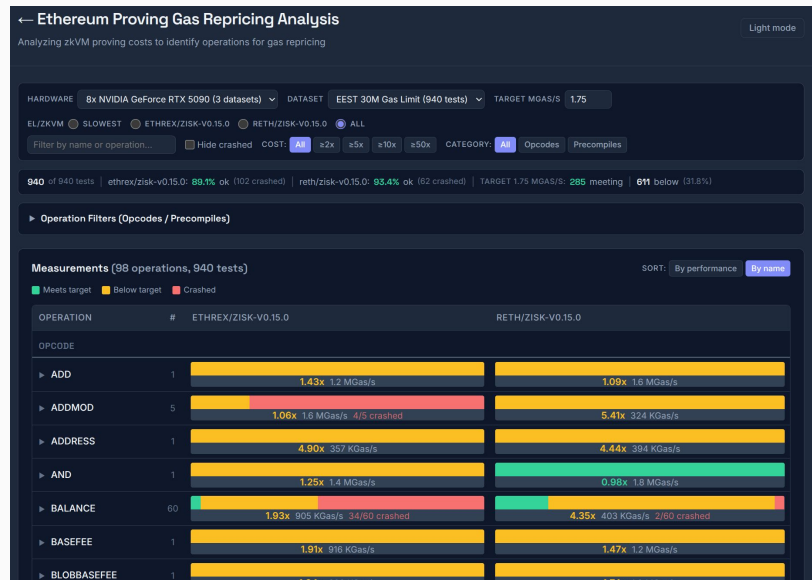
# Project TL;DR

- Protocol repricings blog posts: [Part 1](#) and [Part 2](#)



# Fusaka benchmarks

- Re-run for benchmarks@v0.0.7 (Fusaka)
- Improved [dashboard](#) UI
- Caveat: non-optimized proving cluster
- Next steps: integrate with G\* repricing analysis pipeline



# Proof verification dashboard

- New [Proof verification dashboard](#)
- Metrics:
  - Proof verification latency
  - Proof size
  - Security bits extracted from [soundcalc](#)
- Targets [EIP-7870](#) validator hw specs:

● **attester7870** — INTEL(R) XEON(R) GOLD 6548N · 62 GiB · PassMark ST 3,238 / MT 42,519  
 ▲ **ccx43** — AMD EPYC-Milan Processor · 61 GiB · PassMark ST 2,918 / MT 27,111





# Benchmarkoor custom RPC support

- New feat: configure custom RPC calls after bench runs
- Goal: leverage bloatnet to collect execution witnesses for stateful opcodes

The screenshot displays the Benchmarkoor web interface. At the top, there's a navigation bar with 'Runs' and 'Suites' tabs. The main header shows the suite path: 'Suites / perf-devnet-2-23861500-stateful / 1772115074\_be36477b\_reth'. Below this, a progress bar indicates the status of various components. The main content area features a card for the 'Reth' benchmark run, showing a run ID, a progress indicator (570 / 570), start time (2/26/2026, 11:11:14 AM), and performance metrics: 87.05 MGas/s (88.22 GGas in 16m53s) and 1,140 calls. A 'Test Duration' of 16m54s is also shown. Below the main card, a 'Download List' modal is open, displaying a list of RPC calls under the 'Plain URLs' tab. The list includes 1140 items, with a subset of 1140 'Post-Test RPC Dumps' highlighted. The RPC calls are formatted as JSON, showing the test name, token name, and the specific RPC call being executed, such as 'sload\_sstore[fork\_Osaka-blockchain\_test-10-90-token\_name\_306B\_ERC20-benchmark\_120M].txt/post\_test\_rpc\_calls/eth\_block.json?redirect=true'.